

Authentication and Authorization

Musard Balliu
Lecture 4
DD2395/DD2391

Last Lectures

- Introduction to cryptography
- Security properties of interest
- Symmetric key cryptography
 - OPT, Block cyphers, Hashes, MACs, Authenticated encryption
- Asymmetric key cryptography
 - Problem definition, digital signatures

Questions?

Today's Plan

- **Authentication**
- **Access control**
- **Discretionary Access Control (DAC)**
- **Mandatory Access control (MAC)**

Authentication and Authorization

- **Authentication:** mechanisms that bind principals to actions
- **Authorization:** mechanisms that govern whether actions are permitted
- **Audit:** mechanisms that record and review actions
 - Not covered



Authentication

Classes of Principals

Authentication: mechanisms that bind principals to actions



- Authenticating Humans
- Authenticating Machines

	Humans	Machines
Humans authenticating...	Faces, tickets, passports	Secure attention key, visual secrets
Machines authenticating...	Passwords, biometrics	Tokens, CAs as used in web

Identity

- **Attribute:** property of a principal
 - name, birth date, mother's name
- **Identity:** set of attributes
 - each principal may have many identities of use in different scenarios (student, personal number, athlete)
- **Identifier:** an attribute that is unique within a population
- **Authentication:** Check if an identity matches a set of attributes
 - Example: border guard who checks whether a passport names the bearer
 - Non-example: a video surveillance system attempting to locate suspected terrorists in a crowd
 - this is called identification

Identity enrollement

- **Enrollment:** establishing identity with a system
 - Create an account
 - Get an ID card, visa
 - Register a machine on a network
 - Get a signing key from a provider
- System might (not) verify claimed attributes during enrollment
 - Websites rarely do
 - Governments often do

Authentication of humans to machines

Categories:

- **Something you are**
 - Allow certain of your physical attributes to be measured such that measurements will not be similar for impersonators
 - fingerprint, retinal scan, hand silhouette, a pulse
- **Something you know**
 - Demonstrate knowledge of a secret or fact(s) unlikely to become known to impersonators
 - password, passphrase, PIN, answers to security questions
- **Something you have**
 - Demonstrate possession of some distinctive object that is difficult for an impersonator to obtain or fabricate
 - physical key, ticket, ATM card, token

Authentication of humans to machine

- **Two-factor authentication:** authenticate based on two independent methods
 - ATM card plus PIN
 - password plus registered mobile phone
- **Multi-factor authentication:** two or more independent methods
- **Best to combine separate categories, not reuse categories**
 - non-example: requiring two passwords from a single human: arguably not independent
 - non-example: requiring single password from each of two humans: authenticates two humans then makes *authorization* decision

Passwords

Something you know

Password lifecycle

Password: A secret string a user types in to prove their identity

1. **Create:** user chooses password
 2. **Store:** system stores password with user identifier
 3. **Use:** user supplies password to authenticate
 4. **Change/recover/reset:** user wants or needs to change password
- All steps are tricky and subject to compromise
 - We will only see some of these aspects

Who creates a password?

- **User:** often guessable passwords
- **System:**
 - can produce hard-to-guess passwords (e.g., random ASCII character strings)
 - but users can't remember them
- **Administrator:** reduces to one of the above

Weak passwords

Top 10 passwords 2024, <https://www.purevpn.com/blog/worst-password-list/>

1. 123456
2. 12345678
3. 123456789
4. 12345
5. 1234567
6. password
7. 1password
8. abc123
9. Qwerty
10. 111111

12: iloveyou, 17: princess, 20: football, 27: f***you

Top 20 passwords suffice to compromise 10% of accounts

Strong passwords

- How to characterize strength?
- Difficulty to brute force—"strength" or "security level"
 - if 2^X guesses required, strength is X
- Suppose passwords are L characters long from an alphabet of N characters
 - Then N^L possible passwords
 - Solve for X in $2^X = N^L$
 - Get $X = L \log_2 N$
 - This X is aka **entropy** of password
 - Assuming every password is equally likely (realistic?), X is the *Shannon entropy of the probability distribution*

Entropy of passwords

- Option A:
 - 8 character passwords chosen uniformly at random from 26 character alphabet
 - entropy of $8 \log_2 26 \approx 37$ bits
 - but that means abcdefgh equally likely as ifhslgqz
- Option B:
 - 1 word chosen at random from entire vocabulary
 - average high-school graduate: 50k word vocabulary
 - entropy of $\log_2 50k \approx 16$ bits
- Underlying problem: Shannon entropy not a good predictor of how quickly attackers can crack passwords

Password Storage and Usage

Password storage

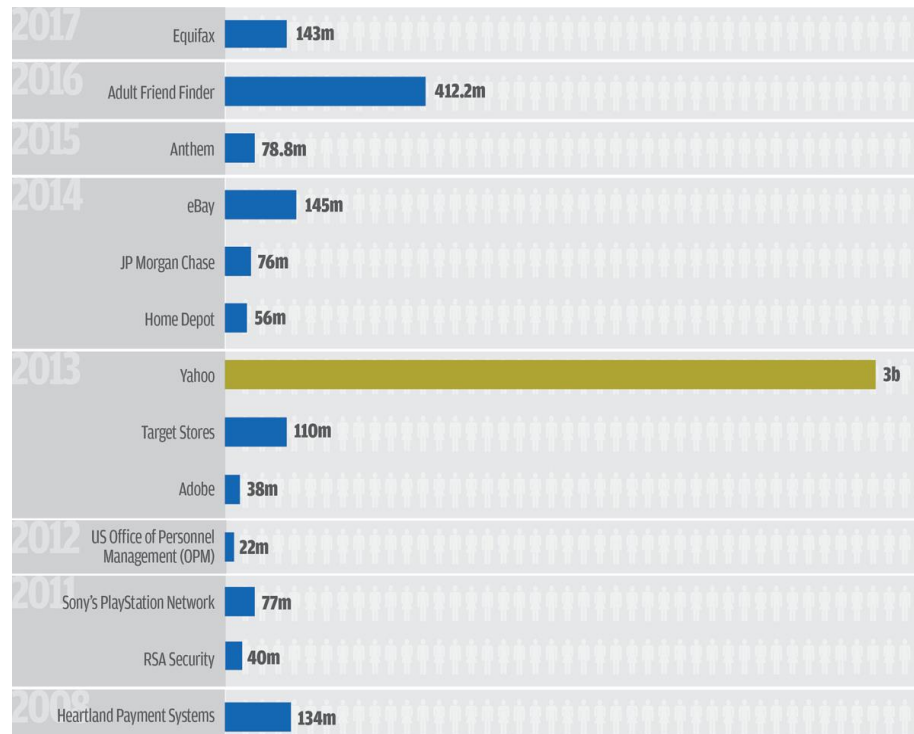
- Password: A secret string a user types in to prove their identity
 - When you create an account with a service: Create a password
 - When you later want to log in to the service: Type in the same password again
- How does the service check that your password is correct?
- Bad idea #1: Store a file listing every user's password
 - Problem: What if an attacker hacks into the service? Now the attacker knows everyone's passwords!
- Bad idea #2: Encrypt every user's password before storing it
 - Problem: The attacker could steal the passwords file *and* the key and decrypt everyone's passwords!
- We need a way to verify passwords *without* storing information that would allow someone to easily recover the original password

Threat Model: Offline Attack

- Adversary can read files from disk



- Adversary can read process memory



Note: users make this worse by reusing passwords across systems.

Password Storage

- **Want:** a function f such that...
 1. easy to compute and store $f(p)$ for a password p
 2. hard given disclosed $f(p)$ for attacker to recover p
 3. hard to trick system by finding password q s.t. $q \neq p$ yet $f(p) = f(q)$
- Encryption would work, but then the key has to live somewhere
- Cryptographic hash functions suffice!
 - one-way property gives (1) and (2)
 - collision resistance gives (3)

Password Hashing

- For each user, store a *hash* of their password
- Verification process
 - Hash the password submitted by the user
 - Check if it matches the password hash in the file
- What properties do we need in the hash?
 - Deterministic: To verify a password, it has to hash to the same value every time
 - One-way: We don't want the attacker to reverse hashes into original passwords

Hashed passwords

- Each user has:
 - username uid
 - password p
- System stores: uid, $H(p)$

To authenticate human Hu to a system S:

1. $Hu \rightarrow S$: uid, p
2. S: let h = stored hashed password for uid;
if h = $H(p)$
then uid is authenticated

Password Hashing: Attacks

- What if two different users decide to use **password123** as their password?
 - Hashes are deterministic: They'll have the same password hash
 - An attacker can see which users are using the same password
- Brute-force attacks
 - Most people use insecure, common passwords
 - An attacker can pre-compute hashes for common passwords: $H(\text{"password123"})$, $H(\text{"password1234"})$, $H(\text{"1234567890"})$, etc.
 - **Dictionary attack**: Hash an entire dictionary of common passwords
 - It works because most passwords chosen by humans are from a relatively small set
 - One dictionary suffices to attack every user
- **Rainbow tables**: An algorithm for computing hashes that makes brute-force attacks easier

Solution: Salted hashed passwords

- Each user has:
 - username uid
 - unique salt s
 - password p
- System stores: uid, s, $H(s, p)$

1. $H_u \rightarrow S$: uid, p

2. S: let h = stored hashed password for uid;
let s = stored salt for uid;
if h = $H(s, p)$
then uid is authenticated

Solution: Salted hashed passwords

- Solution #1: Add a unique, random salt for each user
- **Salt:** A random, public value designed to make brute-force attacks harder
 - For each user, store: username, salt, $H(\text{password} || \text{salt})$
 - To verify a user: look up their salt in the passwords file, compute $H(\text{password} || \text{salt})$, and check it matches the hash in the file
 - Salts should be long and random
 - Salts are not secret
- Brute-force attacks are now harder
 - Assume there are M possible passwords and N users in the database
 - Unsalted database: Hash all possible passwords, then lookup all users' hashes $\Rightarrow O(M + N)$
 - Salted database: Hash all passwords for each user's salt $\Rightarrow O(MN)$
 - Another solution: Pepper – check out the reading material

Threat Model: Online Attack

- Adversary can interact with the system as a user



facebook

Connect with friends and the world around you on Facebook.

Log In

[Forgot password?](#)

Create new account

[Create a Page](#) for a celebrity, brand or business.

When authentication fails

- **Guiding principle:** the system might be under attack, so don't make the attacker's job any easier
- Don't leak valid usernames:
 - Prompt for username and password in parallel
 - Don't reveal which was bad
 - Principle of Failsafe Default
- Record failed attempts and review
 - Perhaps in automated way by administrators
 - Perhaps manually by user at next successful login
 - Principle of Defense in Depth
- Lock account after too many attempts
- Rate limit login

Rate limiting

- **Vulnerability:** hashes are easy to compute
- **Countermeasure:** Slow hashes - hash functions that are slow to compute
 - Slow hash wouldn't bother user: delay in logging hardly noticeable
 - But would bother attacker constructing dictionary: delay multiplied by number of entries
 - Ideally, enough to make constructing a large dictionary prohibitively expensive
- Examples: bcrypt, scrypt, Argon2,...

Slow Hashes

- Cryptographic hashes are usually designed to be fast
 - SHA is designed to produce a checksum of your 1 GB document as fast as possible
- Password hashes are usually designed to be slow
 - Legitimate users only need to submit a few password tries. Users won't notice if it takes 0.0001 seconds or 0.1 seconds for the server to check a password.
 - Attackers need to compute millions of hashes. Using a slow hash can slow the attacker by a factor of 1,000 or more!
 - Note: We are not changing the asymptotic difficulty of attacks. We're adding a large constant factor, which can have a huge practical impact for the attacker

Slowing down fast hashes

- Given a fast hash function...
- Slow it down by iterating it many times:

```
z1 = H(p) ;  
z2 = H(p, z1) ;  
...  
z1000 = H(p, z999) ;  
output z1 XOR z2 XOR ... XOR z1000
```

- Number of iterations is a parameter to control slowdown
 - originally thousands
 - current thinking is 10s of thousands

Slow Hashes: PBKDF2

- **Password-based key derivation function 2 (PBKDF2):** A slow hash function
 - Setting: An underlying function that outputs random-looking bits (e.g. HMAC-SHA256)
 - Setting: The desired length of the output (n)
 - Setting: Iteration count (higher = hash is slower, lower = hash is faster)
 - Input: A password
 - Input: A salt
 - Output: A long, random-looking n -bit string derived from the password and salt
 - Implementation: Basically computing HMAC 10,000 times
- **Benefits (assuming the user password is strong)**
 - Derives an arbitrarily long string from the user's password
 - Output can be directly used as a symmetric key
 - Algorithm is slow, but doesn't use a lot of memory

Authorization

Authorization

- **Authentication:** mechanisms that bind principals to actions
- **Authorization:** mechanisms that govern whether actions are permitted



Access Control Policy

- An **access control policy** specifies which of the **operations** associated with any given **object** each **principal** is authorized to perform
- Expressed as a relation *Auth*:

<i>Auth</i>		Objects	
		dac.tex	dac.pptx
Principals/ Subjects	musard	r,w	r,w
	roberto	r	r
	student		r

Access Control Mechanisms

- A **reference monitor** is consulted whenever one of a predefined set operations is invoked
 - operation $\langle P, O, op \rangle$ is allowed to proceed only if the invoker P is authorized to perform op on object O
- Can enforce **confidentiality** and/or **integrity**
- **Assumption:** Predefined operations are the sole means by which principals can learn or update information.
- **Assumption:** All predefined operations can be monitored (complete mediation).

Who defines authorizations?

- **Discretionary Access Control:** owner defines authorizations
- **Mandatory Access Control:** centralized authority defines authorizations

Discretionary Access Control

Design Principles

- **Principle of Failsafe Defaults** favors defining an access control policy by enumerating privileges rather than prohibitions.
- **Principle of Least Privilege** is best served by having fine-grained principals, objects, and operations.

Protection Domains

- Motivation: users are too coarse-grained to define privileges
- **Protection Domains:**
 - Each thread of control is associated with a protection domain
 - Each protection domain is associated with a different set of privileges
 - We allow transitions from one protection domain to another as execution of the thread proceeds.

Protection Domains

- Typical implementation: certain system calls cause protection-domain transitions.
 - System calls for invoking a program or changing from user mode to supervisor mode are obvious candidates.
- Some operating systems provide an explicit domain-change system call instead
 - the application programmer or a compiler's code generator is then required to decide when to invoke this domain-change system call
- We use the term **attenuation of privilege** for a transition into a protection domain that eliminates privileges.
- We use the term **amplification of privilege** for a transition into a protection domain that adds privileges.

Protection Domains

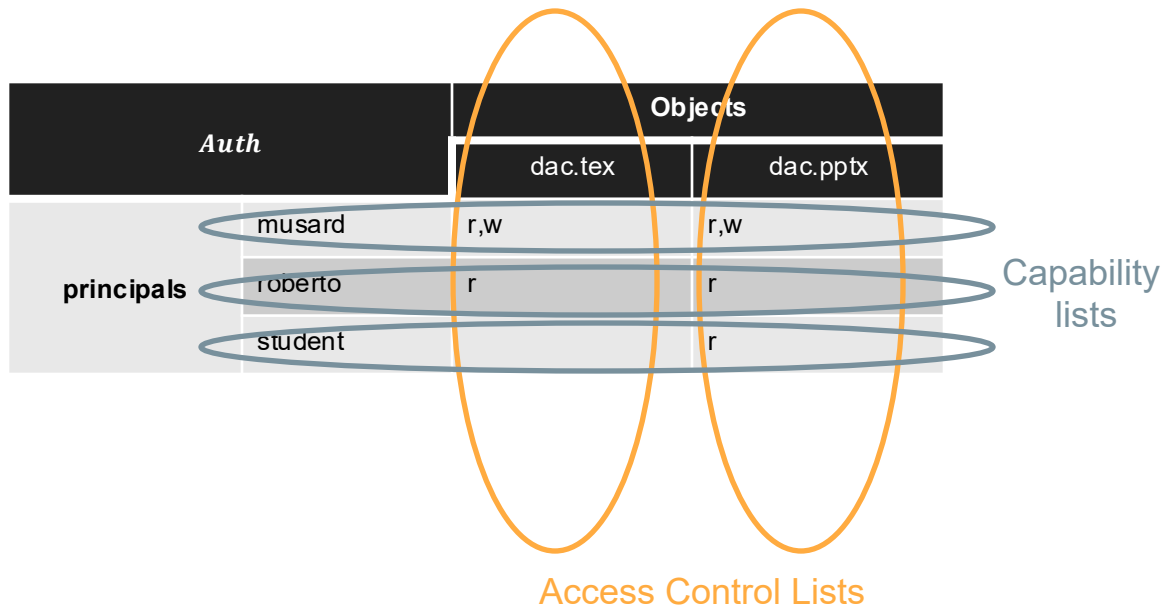
		Objects				
		dac.tex	dac.pptx	musard@sh	musard@edit	musard@powerpoint
principals	musard@sh			e	e	e
	musard@edit	r,w				
	musard@powerpoint		r,w			
	roberto@sh					
	roberto@edit	r				
	roberto@powerpoint		r			
	student@sh					
	student@edit					
	student@powerpoint		r			

Implementing DAC

- Need some way to representing authorization relation (matrix) *Auth*.
- That scheme must support certain functionality:
 - computing whether $\langle P, O, op \rangle \in Auth$ holds and (i.e., whether principal P is authorized to perform operation op on object O ,
 - changing *Auth* in accordance with defined commands
 - associating a protection domain with each thread of control
 - performing transitions between protection domains as execution proceeds.

Instead of Matrices...

- An access control list encodes the non-empty cells associated with a column (object).
- A capability list encodes the non-empty cells associated with a row (principal).



Access Control Lists

- The access control list (ACL) for an object O is a list
$$\langle P_1, Privs_1 \rangle, \langle P_2, Privs_2 \rangle, \dots, \langle P_n, Privs_n \rangle$$
 - Examples: $\langle \text{musard}, \{r, w\} \rangle$, $\langle \text{roberto}, \{r\} \rangle$, $\langle \text{student}, \{r\} \rangle$
- To check whether P_i is allowed to perform op on object O
 - Look up P_i in ACL. If not in the list, reject op
 - Check whether op is in the sent $Privs_i$. If not, reject op .

Access Control Lists

- Advantages:
 - Efficient review of permissions for an object
 - Centralized enforcement is simple to deploy, verify
 - Revocation is straightforward
- Disadvantages:
 - Inefficient review of permissions for a principal
 - Large lists impede performance
 - Vulnerable to confused deputy attack

Confused Deputy

Server: operation(f : file)

S1: buffer := FileSys.Read(f)

S2: results := F(buffer)

S3: chrgs:= calcBill(results)

S4: FileSys.Write(f , results)

S5: FileSys.Write(charges.txt, chrgs)

end Server

- Server handles requests that name a client's file
- Server computes function F on file f, writes the value in results, and records billing information in server's file charges.txt
- Only server holds write privilege for charges.txt
- When server receives f, server is granted read/write privileges on f.
- Expectation: Since client has not write privilege on charges.txt, client cannot modify it.
- Attack: Let f = charges.txt
 - In S4, server has write privileges on charges.txt
- Confused deputy: Server got confused by client request

Confused Deputy: Solution

Server: operation(f : file)

 S1: buffer := FileSys.Read(f)

 S2: results := F(buffer)

 S3: chrgs:= calcBill(results)

 S4: FileSys.Write(f , results)

 S5: FileSys.Write(charges.txt, chrgs)

end Server

- Combine naming with authorization
- Replace names with unforgable bundles (object name + privileges)
- Client conveys a bundle for file f, which server uses to read and update f.
- Server already has a bundle for charges.txt
- Client-supplied bundle for charges.txt does not have write privilege, hence S4 would fail.
- Deputy is not longer confused.

- This is an example of capability.

Capability Lists

- The capability list (C) for a principal P is a list

$\langle O_1, Privs_1 \rangle, \langle O_2, Privs_2 \rangle, \dots, \langle O_n, Privs_n \rangle$

- Examples: $\langle \text{dac.tex}, \{r,w\} \rangle, \langle \text{dac.pptx}, \{r,w\} \rangle$

- Capabilities carry privileges

- **Authorization:** Performing operation op on object O_i requires a principal P to hold a capability $C_i = \langle O_i, Privs_i \rangle$ such that op is in $Privs_i$
- **Unforgability:** Capabilities cannot be counterfeited or corrupted

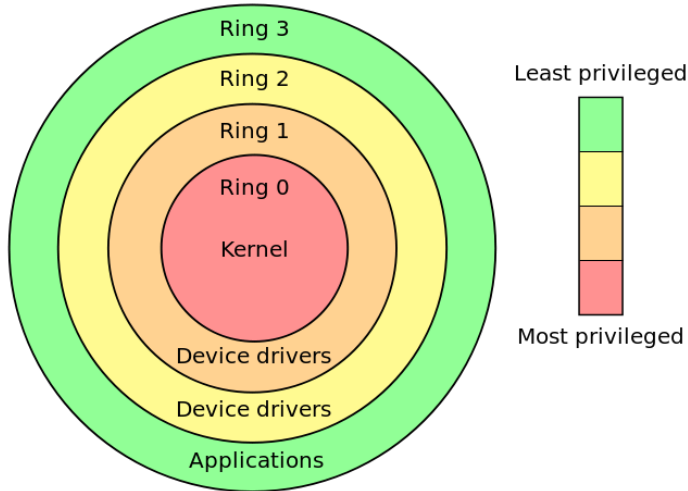
- Capabilities are (typically) transferable

Capabilities

- Advantages:
 - Eliminates confused deputy problems
 - Natural approach for user-defined objects
- Disadvantages:
 - Review of permissions?
 - Delegation?
 - Revocation?

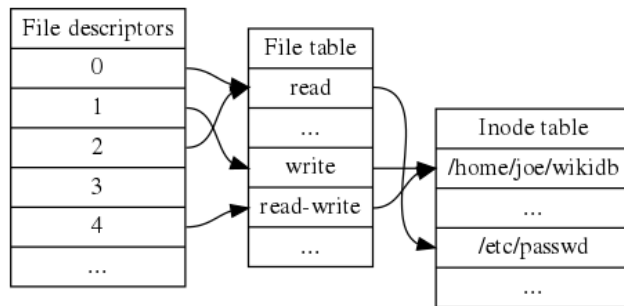
C-Lists

- OS maintains and stores list of capabilities $C_i = \langle O_i, Privs_i \rangle$ for each principal (process)
 - Authorization:** OS mediates access to objects, checks process capabilities
 - Unforgeability:** capabilities are stored in protected memory region (kernel memory)



Example: File Descriptor Table

- In Unix etc, a file descriptor is a handle used to reference files and I/O resources
- File descriptors have modes (read, write) and are stored in per-process file descriptor table
- File descriptors can be passed between processes using `sendmsg()`



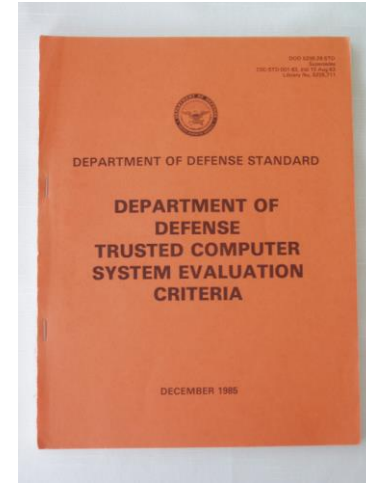
Mandatory Access Control

Summary: DAC

- Discretionary access control (DAC)
 - **Philosophy:** users have the *discretion* to specify policy themselves
 - Commonly, information belongs to the **owner** of object
 - Model: access control authorization **relation**
 - Set of triples (subj,obj,rights)
 - Sometimes described as access control "matrix"
- Implementations:
 - Access control lists (ACLs): each object associated with list of (subject, rights)
 - Capability lists (Privilege lists): each subject associated with list of (object, rights)

Multi-Level Security

- A mechanism for monitoring access control in a system where both principals and objects have security labels drawn from a hierarchy of labels
- Commonly associated with military systems
- Influenced "Orange Book" (DoD Trusted Computer System Evaluation Criteria)
 - A) Verified Protection
 - B) Mandatory Protection
 - C) Discretionary Protection
 - D) Minimal Protection



Sensitivity

- Concern is **confidentiality** of information
- Documents classified according to **sensitivity**: risk associated with release of information
- In US:
 - Top Secret
 - Secret
 - Confidential
 - Unclassified



Compartments

- Documents classified according to compartment(s): categories of information (in fact, aka category)
 - cryptography
 - nuclear
 - biological
 - reconnaissance
- **Need to Know Principle:** access should be granted only when necessary to perform assigned duties (instance of Least Privilege)
 - {crypto, nuclear}: must need to know about **both** to access
 - {}: no particular compartments

Labels

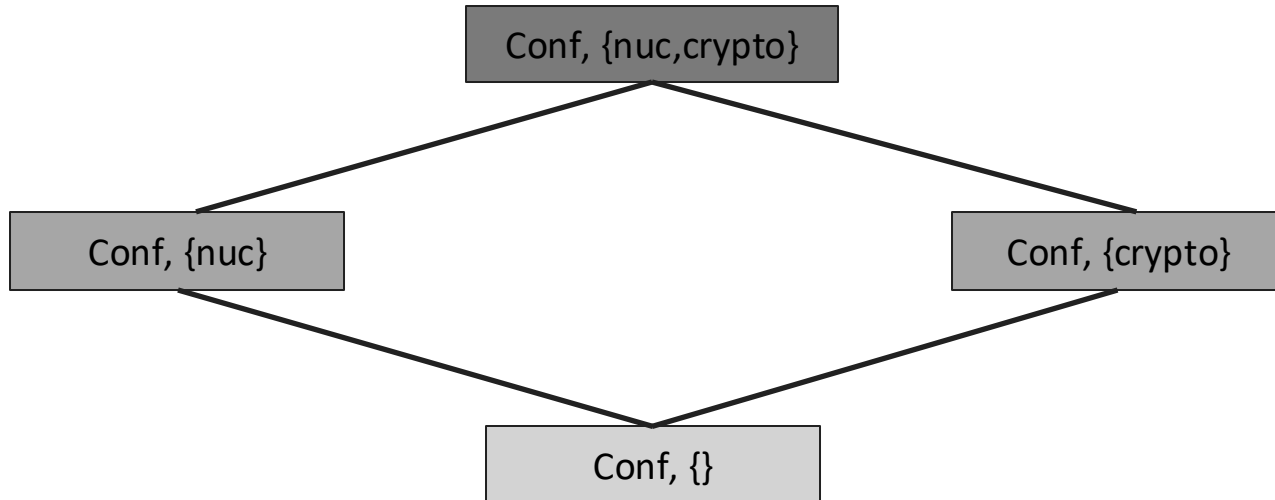
- **Label:** pair of sensitivity level and set of compartments, e.g.,
 - (Top Secret, {crypto, nuclear})
 - (Unclassified, {})
- Users are labeled according to their clearance
- Document is labeled aka classified
 - Perhaps each paragraph labeled
 - Label of document is most restrictive label for any paragraph
- Labels are imposed by organization
- **Notation:** let $L(X)$ be the label of entity X

Restrictiveness of labels

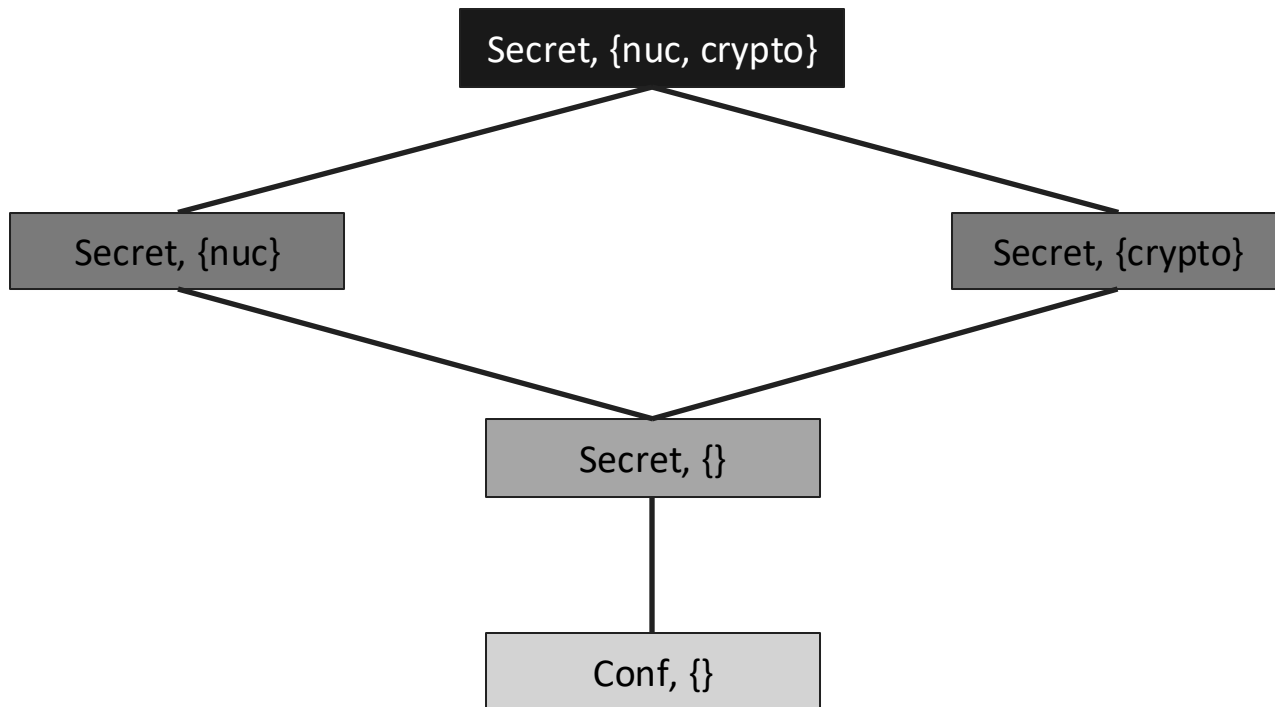
Notation: $L1 \sqsubseteq L2$

- means $L1$ is no more restrictive than $L2$
 - less precisely: $L1$ is less restrictive than $L2$
- **Definition:**
 - Let $L1 = (S1, C1)$ and $L2 = (S2, C2)$
 - $L1 \sqsubseteq L2$ iff $S1 \leq S2$ and $C1 \subseteq C2$
 - Where $\leq$ is order on sensitivity:
Unclassified $\leq$ Confidential $\leq$ Secret $\leq$ Top Secret
- **Examples**
 - $(\text{Unclassified}, \{\}) \sqsubseteq (\text{Top Secret}, \{\})$
 - $(\text{Top Secret}, \{\text{crypto}\}) \sqsubseteq (\text{Top Secret}, \{\text{crypto}, \text{nuclear}\})$

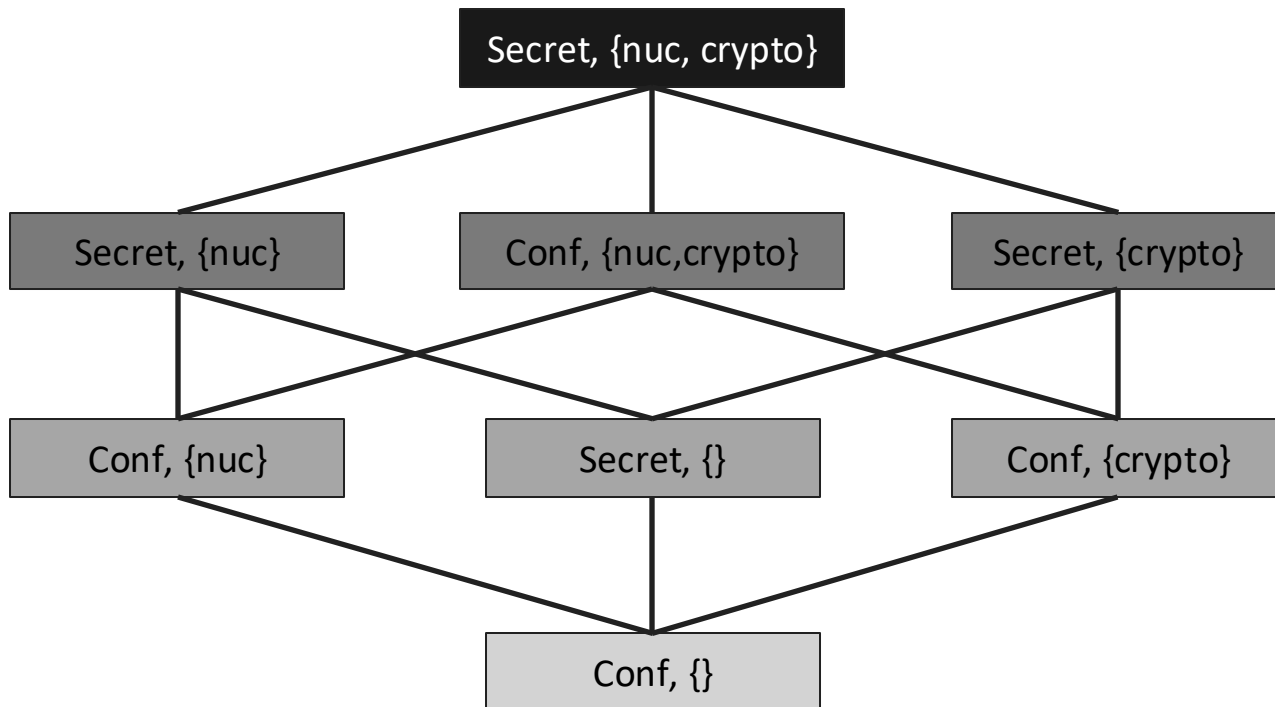
Label partial order



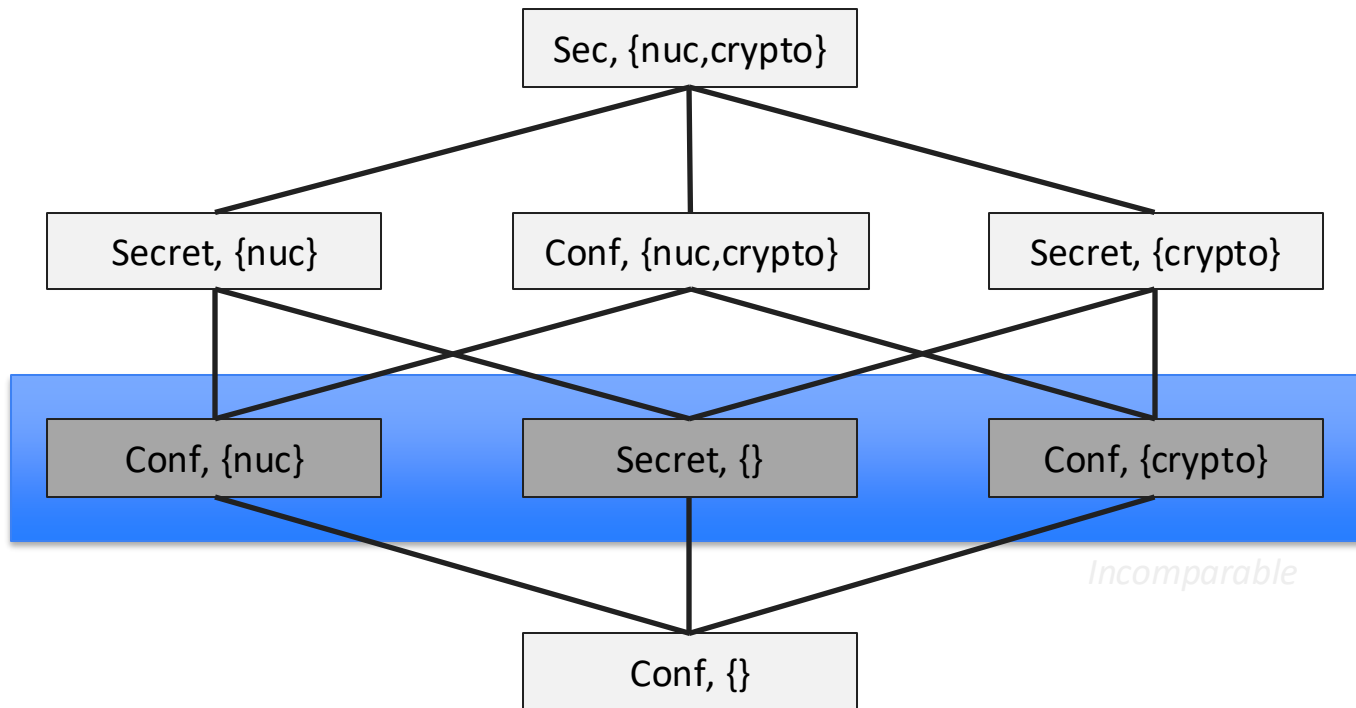
Label partial order



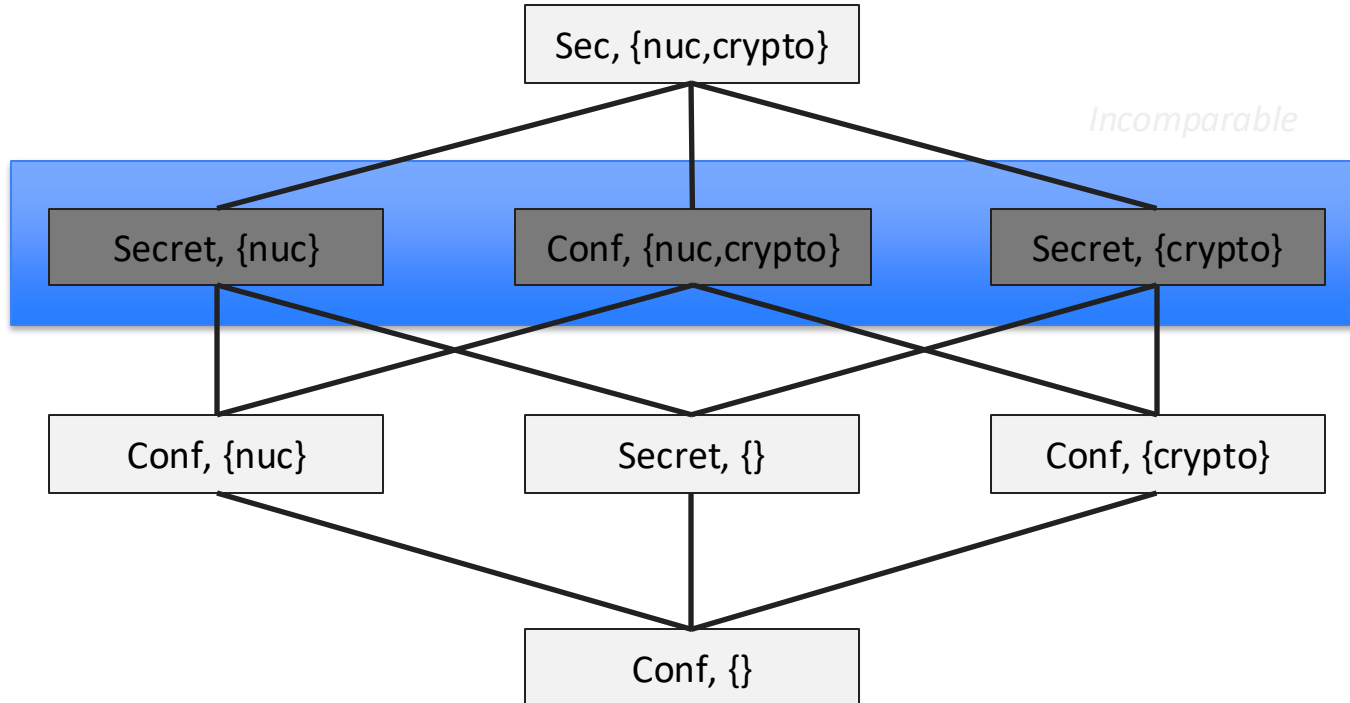
Label partial order



Label partial order



Label partial order



Access control with MLS

- When may a subject read an object?
 - **Threat:** subject attempts to read information for which it is not cleared
 - e.g., subject with clearance Unclassified attempts to read Top Secret information
- When may a subject write an object?
 - **Threat:** subject attempts to *launder* information by writing into a lower-security object
 - e.g., subject with clearance Top Secret reads Top Secret information then writes it into an Unclassified file

Access control with MLS

Threat of concern is **subject** not **user**:

- Users trustworthy by virtue of vetting process for security clearance
- Out of scope (e.g.): user who views Top Secret information and calls *Aftonbladet*
- But still want to enforce Least Privilege
- And malicious programs are a threat...

Trojan Horse



Access control with MLS

- When may a subject read an object?
 - **S may read O** iff $L(O) \sqsubseteq L(S)$
 - object's classification must be below (or equal to) subject's clearance
 - "No read up"
- When may a subject write an object?
 - **S may write O** iff $L(S) \sqsubseteq L(O)$
 - object's classification must be above (or equal to) subject's clearance
 - "No write down"
- Beautiful **symmetry** between these

Reading with MLS

- Scenario:
 - Colonel with clearance (Secret, {nuclear, Europe})
 - DocA with classification (Confidential, {nuclear})
 - DocB with classification (Secret, {Europe, US})
 - DocC with classification (Top Secret, {nuclear, Europe})
- Which documents may Colonel **read**?
 - Recall: S may read O iff $L(O) \sqsubseteq L(S)$
 - DocA: (Confidential, {nuclear}) $\sqsubseteq$ (Secret, {nuclear, Europe})
 - DocB: (Secret, {Europe, US}) $\not\sqsubseteq$ (Secret, {nuclear, Europe})
 - DocC: (Top Secret, {nuclear, Europe}) $\not\sqsubseteq$ (Secret, {nuclear, Europe})

Writing with MLS

- Scenario:
 - Colonel with clearance (Secret, {nuclear, Europe})
 - DocA with classification (Confidential, {nuclear})
 - DocB with classification (Secret, {Europe, US})
 - DocC with classification (Top Secret, {nuclear, Europe})
- Which documents may Colonel **write**?
 - Recall: S may write O iff $L(S) \sqsubseteq L(O)$
 - DocA: (Secret, {nuclear, Europe}) $\not\sqsubseteq$ (Confidential, {nuclear})
 - DocB: (Secret, {nuclear, Europe}) $\not\sqsubseteq$ (Secret, {Europe, US})
 - DocC: (Secret, {nuclear, Europe}) $\sqsubseteq$ (Top Secret, {nuclear, Europe})

Reading and writing with MLS

- Scenario:
 - Colonel with clearance (Secret, {nuclear, Europe})
 - DocA with classification (Confidential, {nuclear})
 - DocB with classification (Secret, {Europe, US})
 - DocC with classification (Top Secret, {nuclear, Europe})
- Summary:
 - DocA: Colonel may read but not write
 - DocB: Colonel may neither read nor write
 - DocC: Colonel may write but not read

Prevention of laundering

- Earlier concern: "subject with clearance Top Secret reads Top Secret information then writes it into an Unclassified file"
- More generally:
 - S reads O1 then writes O2
 - where $L(O2) \sqsubset L(O1)$
 - and regardless of $L(S)$
- **Prohibited by MLS rules:**
 - S read O1, so $L(O1) \sqsubseteq L(S)$
 - S wrote O2, so $L(S) \sqsubseteq L(O2)$
 - So $L(O1) \sqsubseteq L(S) \sqsubseteq L(O2)$
 - Hence $L(O1) \sqsubseteq L(O2)$
 - But combined with $L(O2) \sqsubset L(O1)$, we have $L(O1) \sqsubset L(O1)$
 - Contradiction!
- So access control rules would defeat laundering, Trojan Horse, etc.

Perplexities of writing with MLS

1. **Blind write:** subject may not read higher-security object yet may write it
 - Useful for logging
 - Some implementations prohibit writing up as well as writing down
2. **User** who wants to write lower-security object may not
 - **Attenuation of privilege:** login at a lower security level than clearance
 - Motivated by Trojan Horse
 - Nice (annoying?) application of Least Privilege
3. **Declassification** violates "no write down"
 - Encryption or billing procedure produces (e.g.) Unclassified output from Secret information
 - Traditional solution is **trusted subjects** who are not constrained by access control rules

MLS in OSs

SELinux

- Kernel security module, dates back to NSA c. 2000, merged with Linux kernel mainline in 2.6
- Goal: separate security policy from security decisions
- Supports mandatory access controls in reference policy. When MLS is enabled:
 - Each principal (user or process) is assigned a context (username, role, domain, (sensitivity))
 - Each object (file, port, hardware) is assigned a context
 - SELinux enforces MLS
 - Android supports SELinux



Formalizing MLS

[Bell and LaPadula 1973]

- Formal mathematical model of MLS plus access control matrix
- Proof that information cannot leak to subjects not cleared for it
- "No read up": simple security property
- "No write down": *-property
- *"The influence of [BLP] permeates all policy modeling in computer security" –*

Matt Bishop

- Influenced Orange Book
- Led to research field "foundations of computer security"

BLP, for integrity

- BLP is about confidentiality
- Adapted to integrity by Biba [1977]: same rules, different lattice
 - Instead of Unclassified and Secret, labels could be Untrusted and Trusted
- $L1 \sqsubseteq L2$ means “L1 may flow to L2 without breaking confidentiality”
 - BLP: low secrecy sources may flow to high secrecy sinks
 - Hence Unclassified $\sqsubseteq$ Secret, but not vice versa
 - Biba: low integrity sources may not flow to high integrity sinks
 - Hence Trusted $\sqsupseteq$ Untrusted, but not vice versa

Biba model

- **S may read O iff $L(O) \sqsubseteq L(S)$**
 - E.g., Trusted subject cannot read Untrusted object
 - But Untrusted subject may read Trusted object
- **S may write O iff $L(S) \sqsubseteq L(O)$**
 - E.g., Trusted subject may write Untrusted object
 - But Untrusted subject may not write Trusted object

Beyond Multi-level Security...

Mandatory access control comes in many different forms (not just MLS):

1. Brewer-Nash (consulting firm)
2. Clark-Wilson (business)
3. Role-based access control (organization)
4. Clinical information systems (medicine)

Homework

Check out the reading material for Lecture 4 and read the suggested articles